Comparing the Accuracy and Computational Cost of Numerical and Neural Network Methods at Approximating Non-linear Autocatalytic Reactions

June Kim (1011280604), Jacky Li (1011271678), Junhee Park (1011120984), Brian Ye (1010834493)

University of Toronto Engineering Science, MAT292

December 19th, 2025

Abstract

Numerical methods are compared with a physics-informed neural network (PINN) for solving nonlinear autocatalytic ordinary differential equations. The Brusselator model, an autocatalytic chemical oscillator, is used as a benchmark system due to its dynamical behavior. Euler's method, the improved Euler (Heun) method, and the fourth-order Runge–Kutta (RK4) method are implemented and evaluated across several time steps and parameter values. Their performance is assessed in terms of accuracy, stability, and computational efficiency using a high-resolution RK4 solution ($\Delta t = 10^{-3}$) as a reference.

Results show that RK4 achieves mean-squared errors several orders of magnitude lower than Euler-based methods for comparable time steps, while maintaining stability at time steps up to two orders of magnitude larger. In contrast, Euler's method requires very small step sizes to achieve acceptable accuracy, resulting in significantly higher computational cost. The PINN demonstrates the ability to approximate global solution behavior, but its performance strongly depends on qualitative characteristics of the autocatalytic reaction, performing poorly in stiff and highly oscillatory regimes where phase errors and oscillations dominate. Although PINN inference is computationally efficient after training, the total training time limits its practical advantage for this problem.

Overall, this study highlights the robustness and reliability of classical numerical solvers for autocatalytic systems and clarifies the conditions under which physics-informed neural networks may or may not offer advantages over traditional time-stepping methods.

Contents

1 Introduction

1.1 Autocatalytic Reactions

Autocatalytic reactions involve one or more of the products re-entering the process as reactants to facilitate the formation of additional products. This feedback mechanism leads to nonlinear kinetics and is a defining characteristic of many chemical, biological, and physical systems [1]. Thus, autocatalytic reactions are modeled using nonlinear differential equations.

In chemistry, autocatalytic reactions describe chain reactions such as the Belousov–Zhabotinsky oscillator [2], in which product feedback sustains and amplifies oscillatory behavior. In biology, these mechanisms are found in models of enzymes and self-replicating molecules, critical to understanding processes of DNA replication and pathogen spread. They also occur in ecology, where cooperative interactions within a population promote further growth [2].

These reactions often lack exact solutions and require numerical methods. However, the strong nonlinearities and feedback present in autocatalytic systems pose significant challenges for time-integration methods. Such systems may exhibit oscillations, sharp transients, or long-term sensitivity to numerical error, making them particularly demanding for low-order or explicit schemes [2]. Consequently, autocatalytic reaction models provide a rigorous testbed for evaluating the accuracy, stability, and computational efficiency of various solvers.

1.2 Modelling Approach and Objectives

The objective is to compare the performance of numerical time-integration methods with a physics-informed neural network (PINN) when applied to nonlinear autocatalytic systems. The focus is on assessing how accurately and efficiently these approaches resolve the dynamics of such systems, particularly in regimes characterized by strong nonlinearity and sustained oscillatory behavior.

The Brusselator is a theoretical model of an autocatalytic chemical oscillator and is will be used as a benchmark in this study of nonlinear dynamics due to its simplicity, analytical tractability, and rich qualitative behavior [3]. Its governing equations exhibit steady states, limit cycles, and bifurcation phenomena depending on parameter values, making it well suited for evaluating the stability and accuracy of solvers [3]. See Appendix A for more information.

Three classical numerical methods are considered: Euler's method, the improved Euler method (Heun's method), and the fourth-order Runge–Kutta (RK4) method. These methods span a range of orders of accuracy and computational complexity, providing insight into the trade-offs between simplicity, stability, and error control.

In parallel, a physics-informed neural network approach is implemented to solve the same system. Unlike traditional numerical solvers, PINNs approximate the solution by training a neural network to minimize a loss function that incorporates the governing differential equations, initial conditions, and physical constraints. This enables the model to leverage both data-driven approximation and known physical structure, offering a fundamentally different paradigm for solving differential equa-

tions. The primary objectives of this study are to a) evaluate the accuracy of Euler, improved Euler, and RK4 to PINN solutions, b) to compare the computational cost and efficiency of each approach, and c) to assess the ability of each method to capture key qualitative features of the Brusselator dynamics, including oscillatory behavior and long-term stability.

Through this comparative analysis, the report aims to provide insight into the suitability of traditional numerical solvers and physics-informed neural networks for modeling autocatalytic reaction dynamics, and to clarify the contexts in which each approach may be most effective.

2 Mathematical Background

2.1 Numerical Methods

The three numerical methods employed include EM, IEM, and RK4. As these concepts have been covered in the course, the explicit equations and more detailed mathematical background of these methods are left in Appendix B.

From the Appendix, note the following: Euler's method has a local truncation error of order $\mathcal{O}(h^2)$ and a global error of order $\mathcal{O}(h)$ [4]. Improved Euler's method yields a local truncation error of order $\mathcal{O}(h^3)$ and a global error of order $\mathcal{O}(h^2)$ [4]. Both Euler's and improved Euler's methods are explicit schemes and therefore simple to implement, yet remain sensitive to step size selection and may require very small time steps to accurately capture oscillatory or stiff dynamics, limiting their efficiency for complex autocatalytic systems. RK4 achieves a local truncation error of order $\mathcal{O}(h^5)$ and a global error of order $\mathcal{O}(h^4)$ [4]. RK4 is significantly more accurate than EM or IEM for a given step size, yet requires four function evaluations per time step, resulting in a higher computational cost. Additionally, RK4 remains an explicit method and may encounter stability limitations when applied to stiff systems.

2.2 Physics Informed Neural Networks (PINNs)

PINNs are a class of neural networks designed to solve differential equations by incorporating the underlying physics directly into the training process. Instead of just learning from data, PINN approximates the solution u(t,x) of a differential equation and minimizes a loss function that penalizes deviations from the governing equations and initial conditions [5]. For the context of this project, a modified loss function consisting of the weighted average between physics loss, initial condition loss, and data loss will be used to penalize the model. The respective weights for physics, initial condition, and data loss are 1, 100, and 50. A greater emphasis is placed on initial condition loss because inaccurate initial predictions will propogate through time and lead to complete divergence from the expected solution [6]. In addition, data loss is weighed more than physics loss, because high frequency solutions are more difficult to capture with physics loss and an explicit comparison to the analytical solution must be made [6].

Loss Type	Description	Mathematical Formula
Physics Loss	Enforces that the model's predictions satisfy the Brusselator ODE.	$\mathcal{L}_{\text{physics}} = \text{MSE}\big(f(\hat{x}, \hat{y}, t)\big)$
Initial Condition Loss	Penalizes deviations of the model's output from the initial conditions.	$\mathcal{L}_{\text{IC}} = \text{MSE}\big(u(0) - u_0\big)$
Data Loss	Penalizes difference between model prediction and analytical solution.	$\mathcal{L}_{\text{data}} = \text{MSE}\big(u(t_i) - y_i\big)$

Table 1: Main loss functions. Here, u represents the model solution, f is the residual of the Brusselator ODEs, u_0 the initial condition, and y_i analytical data points at t_i.

PINNs use automatic differentiation to compute derivatives of the network output with respect to inputs, which allows direct evaluation of differential operators. The training process is then standard neural network optimization (e.g., gradient descent) based on physics-informed and other losses. This approach unifies data-driven modeling with first-principles physics, enabling solutions for problems where other methods may be slow or noisy [6].

3 Methodology

Euler's method was implemented to solve the Brusselator model using Python, with `NumPy` for numerical computation and `Matplotlib` for visualization, followed by IEM and RK4.

A PINN was then developed to solve the Brusselator model. The PINN was initially trained in an unsupervised manner, where the loss function consisted of mean squared error terms enforcing the governing differential equations and the initial conditions. After limitations in accuracy were observed, the training approach was extended to include supervised learning, using solution data generated by the RK4 method. The key innovation in the PINN architecture was the use of a continuous function across the parameter space. The network contained ≈400,000 parameters, used a GELU activation function, and a Xavier normal weight initialization. The loss function was a combination of a physics loss ($\lambda = 1.0$), initial condition loss ($\lambda = 100.0$), and a data loss ($\lambda = 50.0$). (Fig. 2).

Once results were obtained from Euler's method, improved Euler's method, RK4, and the PINN, a Python-based graphical user interface (GUI) was created to run inference and visualize solutions. Using this interface, all four approaches were compared against a high-accuracy reference solution (see Appendix C for a GUI guide). The comparison focused on solution accuracy, computational cost, and qualitative behavior such as the ability to reproduce the oscillatory dynamics of the Brusselator system.

Initially, single comparisons were made, where one trial run for each method was compared with different parameters and time steps for numerical methods. However, drastic differences in output each time the trial was ran became apparent, motivating the need for a more broad comparison. Hence, a global comparison was created, and is accessible in the GUI. This global comparison runs

each method thousands of times, with randomly generated parameters each trial, and outputs the average and standard results in terms of accuracy and time taken.

4 Results

4.1 Time-Series Results

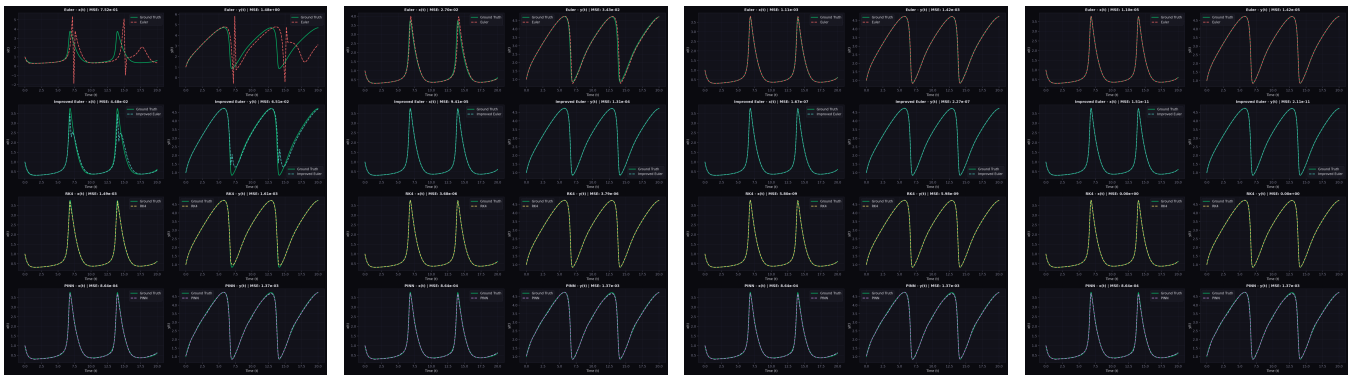
Local comparisons (single trial) were conducted for the Brusselator model with parameters $A = 1$, $B = 3$, $x_0 = 1$, and $y_0 = 1$ over the interval $t \in [0, 20]$. An RK4 solution with $\Delta t = 0.001$ was used as the ground truth. Numerical methods were evaluated at decreasing time steps to assess convergence behavior and solution accuracy, while the Physics-Informed Neural Network (PINN) used a fixed temporal discretization of 1000 points.

At a coarse step size of $\Delta t = 0.2$ (Fig. 1 a), Euler's method performs poorly, yielding an RMSE of 1.06 and a total MSE of 1.12, indicating severe numerical instability. Improved Euler significantly reduces error (RMSE = 0.234), while RK4 further improves accuracy (RMSE = 0.0394). The PINN achieves an RMSE of 0.0334, comparable to RK4 in magnitude.

Reducing the time step to $\Delta t = 0.05$ (Fig. 1 b) leads to rapid convergence of RK4, achieving an RMSE of 1.93×10^{-3}. Improved Euler also converges well (RMSE $= 1.06 \times 10^{-2}$), while Euler's method remains comparatively inaccurate (RMSE = 0.175).

At $\Delta t = 0.01$ (Fig. 1 c), RK4 achieves near machine-precision accuracy with RMSE $= 7.7 \times 10^{-5}$, while Improved Euler attains RMSE $= 4.43 \times 10^{-4}$. Euler's method continues to lag behind, requiring substantially smaller time steps to reach comparable accuracy.

At the finest resolution $\Delta t = 0.001$ (Fig. 1 d), RK4 is indistinguishable from the ground truth (RMSE ≈ 0). Improved Euler also converges strongly (RMSE $= 4 \times 10^{-6}$), while Euler achieves acceptable accuracy only at the cost of significantly increased runtime. The PINN exhibits a persistent RMSE of 0.0334, as its error is dominated by model limitations rather than temporal discretization [6].



(a) $\Delta t = 0.2$ (b) $\Delta t = 0.05$ (c) $\Delta t = 0.01$ (d) $\Delta t = 0.001$

Figure 1: Time-series comparisons of numerical methods against the RK4 ground truth for decreasing time steps.

4.2 PINN Training Behavior and Limitations

The PINN was trained using 5000 parameter sets with 1000 validation cases over $t \in [0, 20]$. The best validation loss of 0.590 occurred at epoch 63,116 out of 113,116 total epochs. After this point, training loss continued to decrease (final training loss = 0.838) while validation loss increased (final validation loss = 3.63), indicating overfitting.

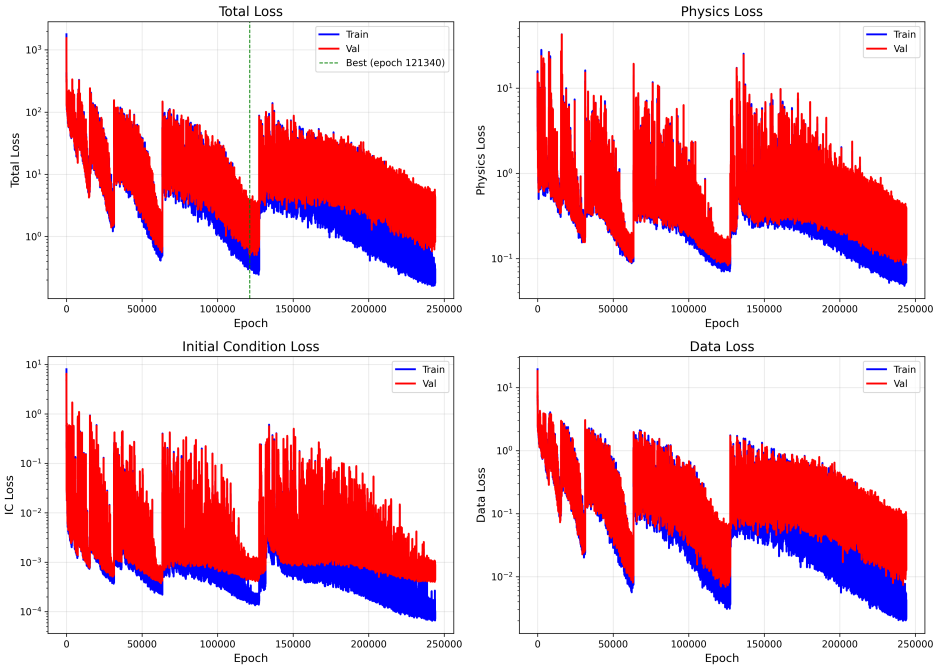


Figure 2: PINN training loss history: total, physics, initial condition, and data losses.

Multiple descents in the loss curves, shown in Fig. 2, are attributed to cosine annealing, which facilitates navigation of local minima in the loss landscape. Near the end of training, validation loss began decreasing again, suggesting that additional training could further improve performance [7]; however, project time constraints prevented further optimization.

4.3 Global Error Statistics

Global performance was evaluated using 5000 trials and randomized parameter sets with $A \in [0.5, 2.5]$, $B \in [1.0, 6.0]$, and $x_0, y_0 \in [0, 3]$. Numerical methods were evaluated at $\Delta t = 0.005$, while the PINN was evaluated using fixed inference.

RK4 achieved the lowest average error with an average MSE of 6.34×10^{-8} and a standard deviation of 3.28×10^{-7}. Improved Euler achieved an average MSE of 3.05×10^{-6}, while Euler's method exhibited a substantially higher average MSE of 5.32×10^{-3} with large variance. The PINN demonstrated an average MSE of 9.24×10^{-3} with the largest standard deviation among all methods (3.81×10^{-2}), confirming strong sensitivity to parameter regimes. The global test depicts large standard deviations, implying accuracy for specific situations and not for others. This also shows that a Global Average Error may not be the best comparison metric due extremely high variance between different sets of parameters.

5 Comparison and Discussion

5.1 Accuracy Across Regimes

Figure 3 illustrates the global speed–accuracy tradeoff.

Figure 3: Global speed–accuracy tradeoff across all methods.

RK4 consistently provides the best balance between accuracy and computational cost (figure 3), achieving errors several orders of magnitude lower than Euler's method while permitting significantly larger time steps. Improved Euler occupies an intermediate regime, offering acceptable accuracy for moderate time steps but degrading rapidly otherwise.

The PINN performs competitively only in select parameter regimes and exhibits high variance globally. This explains the discrepancies observed between local and global tests and motivates regime-specific analysis rather than relying on isolated case studies. Hence, a key outcome of both the local and global comparisons is that the relative performance of each method is highly dependent on the qualitative behavior of the Brusselator dynamics for a given parameter set and initial condition.

5.2 Computational Cost and Resources

Runtime distributions are shown in Fig. 4.

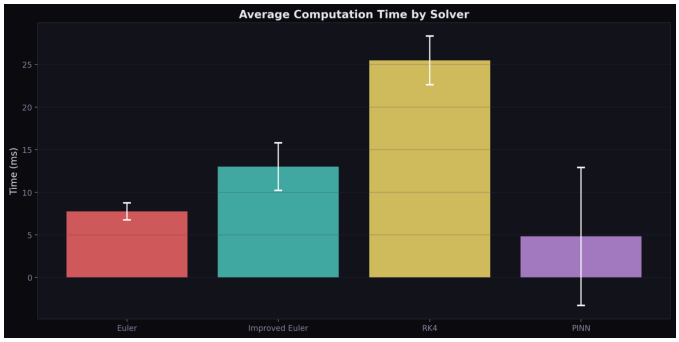


Figure 4: Runtime distribution across 5000 randomized trials.

Euler's method is fastest per step but inefficient overall due to slow convergence. RK4 incurs higher per-step cost but achieves superior accuracy with fewer steps, resulting in competitive total runtimes (figure 4). For example, at $\Delta t = 0.05$, RK4 achieves an RMSE nearly two orders of magnitude smaller than Euler's method while requiring less than four times the runtime. PINN inference is computationally inexpensive as it demonstrated the lowest average computation time; however, training required approximately **7 days** of computation. All reported PINN runtimes exclude training time, emphasizing the significant upfront cost associated with neural-network-based solvers.

5.3 Qualitative Behaviours of the PINN

To address the observation that the PINN model performance depended strongly on the dynamical regime of the Brusselator, several values of the parameters A, B, and initial conditions were tested.

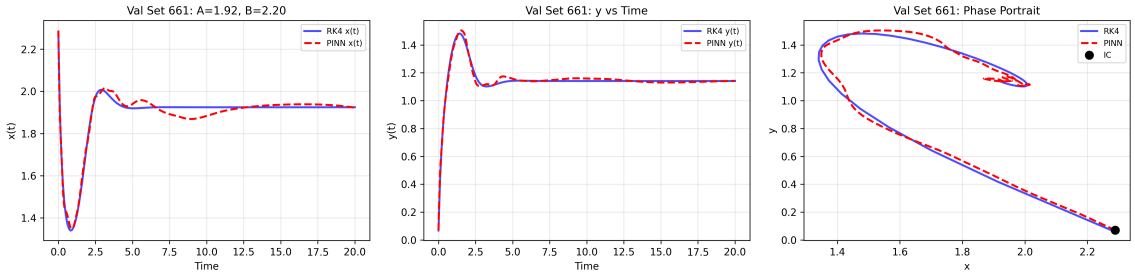


Figure 5: Stiff and unchanging behaviour with time as the PINN tries to predict smooth osciallations and curves, from A = 1.92, and B = 2.20

In stiff regimes, where the solution varies slowly in time, classical numerical solvers consistently outperformed the PINN, which tended to introduce spurious oscillations not present in the true solution as seen in Figure 5.

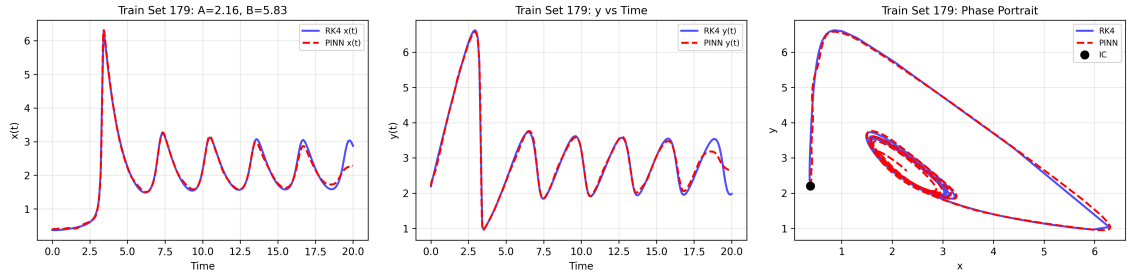


Figure 6: High frequency solutions with deterioration near the final few oscillations, from A = 2.16 and B = 5.83

Conversely, in highly oscillatory regimes involving rapid amplification or decay, the PINN struggled to resolve high-frequency dynamics, leading to significant phase and amplitude errors, as seen in Figure 6. These observations highlight that PINN performance is highly regime-dependent, whereas classical time-stepping methods remain robust when appropriate step sizes are chosen.

6 Conclusion

For the Brusselator benchmark considered here, classical numerical integration methods remain the most practical option. In particular, RK4 achieves the best accuracy for this low-dimensional, non-stiff system while remaining computationally inexpensive. Improved Euler provides a clear and inexpensive accuracy advantage over forward Euler for the same step sizes. Physics-informed neural networks offer qualitative advantages: they produce smooth solutions, can incorporate physical priors directly, and can be extended to learn across parameter families or for inverse problems [7]. However, for this two-dimensional ODE the PINN required substantial training time to approach RK4-level accuracy and did not outperform RK4 on a per-trajectory basis. Therefore, unless one needs a single trained model that will be used for many forecasts, parameter sweeps, or inverse tasks where data is scarce, classical solvers such as RK4 are the recommended approach.

References

[1] A. K. Horváth, "Correct classification and identification of autocatalysis," *Physical Chemistry Chemical Physics*, vol. 23, no. 12, pp. 7178–7189, 2021, doi: 10.1039/D1CP00224D.

[2] A. I. Hanopolskyi, V. A. Smaliak, A. I. Novichkov, and S. N. Semenov, "Autocatalysis: Kinetics, mechanisms and design," *ChemSystemsChem*, vol. 3, no. 1, Art. no. e2000026, 2021, doi: 10.1002/syst.202000026.

[3] M. Schober, T. A. Nieminen, P. Hennig, and S. Hirche, "A probabilistic model for the numerical solution of initial value problems," *Statistics and Computing*, vol. 29, pp. 99–122, 2019, doi: 10.1007/s11222-017-9798-7.

[4] J. R. Brannan and W. E. Boyce, *Differential Equations: An Introduction to Modern Methods and Applications*, 3rd ed. Hoboken, NJ, USA: John Wiley & Sons, 2015.

[5] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Scientific machine learning through physics-informed neural networks: Where we are and what's next," *Journal of Scientific Computing*, vol. 90, no. 1, 2022, doi: 10.1007/s10915-022-01939-z.

[6] D. Chen, Z. Pan, and H. Ma, "Physics-informed neural networks for high-frequency and multiscale problems," *arXiv preprint* arXiv:2401.02810, 2024. [Online]. Available: `https://arxiv.org/abs/2401.02810`

[7] M. Raissi, P. Perdikaris, and G.E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," Journal of Computational Physics, 2019.

[8] M. P. McDowell, "Mathematical modeling of the Brusselator," prepared for J. M. Powers, Department of Aerospace and Mechanical Engineering, University of Notre Dame, Notre Dame, IN, USA, Feb. 6, 2008. [Online]. Available: `https://academicweb.nd.edu/~powers/mcdowell.pdf`.

Appendix A: The Brusselator

The Brusselator is a theoretical model of a chemical oscillator, proposed by Ilya Prigogine to illustrate how simple chemical reactions can produce sustained oscillations rather than settling to equilibrium [8]. It describes two dynamic species, X and Y, interacting through nonlinear reactions with constant inputs A and B [8]. The system's behaviour is governed by the equations

$$\frac{dx}{dt} = A - (B+1)x + x^2 y, \qquad \frac{dy}{dt} = Bx - x^2 y, \tag{1}$$

which define how the concentrations of X and Y evolve over time.

At steady state, $\frac{dx}{dt} = 0$ and $\frac{dy}{dt} = 0$. When

$$\frac{d}{dx}\left(\frac{dx}{dt}\right) < 0,$$

the steady state is stable and the system converges to constant concentrations. However, when

$$\frac{d}{dx}\left(\frac{dx}{dt}\right) > 0,$$

the steady state becomes unstable and the system exhibits limit-cycle oscillations, meaning the concentrations of X and Y continually rise and fall in a repeating cycle. Although the Brusselator does not represent any specific real chemical system, it remains an important model for understanding nonlinear dynamics, self-organization, and pattern formation in chemical and biological systems [8].

The Brusselator is an excellent model for studying autocatalytic reactions and testing numerical or neural approximation methods because it captures complex nonlinear behaviour within a simple mathematical structure. Its key autocatalytic term, x^2y, represents self-amplification, enabling oscillations and self-organization similar to those observed in real chemical systems. Despite consisting of only two coupled differential equations, it exhibits rich dynamics such as steady states, limit cycles, and bifurcations, making it ideal for testing numerical solvers and stability analyses [3]. Because its behaviour is well understood, it also serves as a benchmark for modern data-driven approaches such as neural ODEs and physics-informed neural networks, which aim to learn and predict nonlinear dynamics directly from data. For these reasons, the Brusselator model will be used as a benchmark to compare several methods for approximating solutions to autocatalytic reactions.

Appendix B: Numerical Methods Background

Euler's and Improved Euler's Method

Euler's method is based on a first-order Taylor expansion of the solution and approximates the solution by locally extrapolating along straight-line segments [4]. To apply Euler's method, the

ordinary differential equation must be written in explicit form,

$$\frac{dy}{dt} = f(y,t),$$

and an initial condition (t_0, y_0) lying on the solution curve must be specified. Given a step size $h > 0$, Euler's method advances the numerical solution by assuming that the slope remains constant over each time interval. Starting from (t_n, y_n), the approximation is updated according to

$$y_{n+1} = y_n + h\, f(y_n, t_n).$$

This approach is computationally inexpensive and straightforward to implement; however, its accuracy is limited. Euler's method has a local truncation error of order $\mathcal{O}(h^2)$ and a global error of order $\mathcal{O}(h)$ [4], meaning that small step sizes are required to achieve reliable results.

Improved Euler's method, also known as Heun's method, enhances Euler's method by incorporating information from both the beginning and the end of each time step. The method first computes a provisional value using Euler's method,

$$y_n^* = y_n + h\, f(y_n, t_n),$$

and then updates the solution by averaging the slopes at (t_n, y_n) and $(t_n + h, y_n^*)$:

$$y_{n+1} = y_n + \frac{h}{2}\left[f(y_n, t_n) + f\left(y_n^*, t_n + h\right)\right].$$

This predictor–corrector structure yields a local truncation error of order $\mathcal{O}(h^3)$ and a global error of order $\mathcal{O}(h^2)$ [4], providing a substantial improvement in accuracy with only a modest increase in computational cost.

Both Euler's and improved Euler's methods are explicit schemes and therefore simple to implement. Both methods remain sensitive to step size selection and may require very small time steps to accurately capture oscillatory or stiff dynamics, limiting their efficiency for complex autocatalytic systems.

Runge–Kutta Fourth-Order (RK4) Method

The fourth-order Runge–Kutta method is a widely used explicit time-integration scheme that further improves numerical accuracy over EM and IEM. Like the improved Euler method, which averages slope information at the beginning and end of each time step, RK4 incorporates multiple slope evaluations within each interval to obtain a more accurate approximation of the solution [4]. Consider an initial value problem of the form

$$\frac{dy}{dt} = f(y,t), \qquad y(t_0) = y_0.$$

Given a step size $h > 0$, RK4 computes four intermediate slopes: k_1 is the slope at the beginning of the interval; k_2 and k_3 approximate slopes at the midpoint using previous estimates; and k_4 is the slope at the end of the interval. These slopes are defined as

$$\begin{aligned} k_1 &= hf(y_n, t_n), & k_2 &= hf\left(y_n + \tfrac{k_1}{2},\, t_n + \tfrac{h}{2}\right), \\ k_3 &= hf\left(y_n + \tfrac{k_2}{2},\, t_n + \tfrac{h}{2}\right), & k_4 &= hf(y_n + k_3,\, t_n + h). \end{aligned}$$

The numerical solution is then advanced using the weighted average

$$y_{n+1} = y_n + \frac{1}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right).$$

By incorporating slope information from multiple points within each time step, RK4 achieves a local truncation error of order $\mathcal{O}(h^5)$ and a global error of order $\mathcal{O}(h^4)$ [4]. This fourth-order accuracy allows RK4 to maintain high solution fidelity even with moderately large step sizes, making it significantly more accurate than Euler's and improved Euler's methods for a given step size.

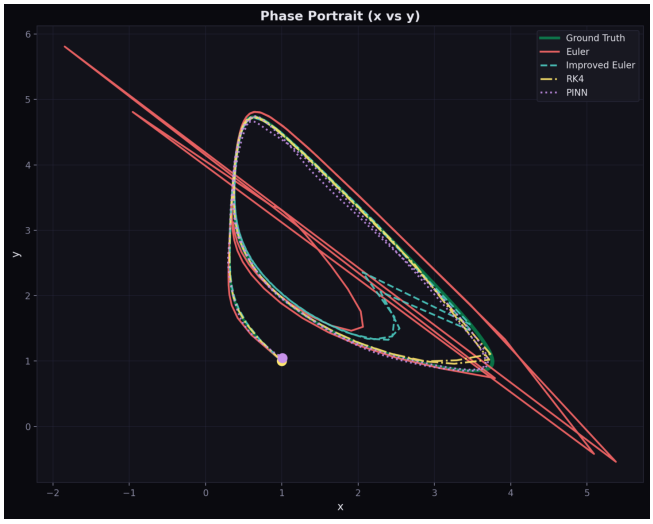
Despite its accuracy, RK4 requires four function evaluations per time step, resulting in a higher computational cost than lower-order methods. Additionally, RK4 remains an explicit method and may encounter stability limitations when applied to stiff systems.

Appendix C: GUI Guide

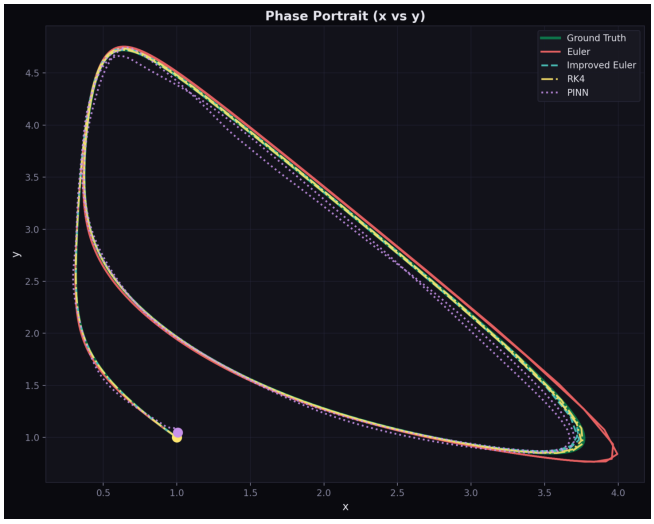
The GUI can be accessed at https://brusselator.streamlit.app/. Use the configuration panel on the left side to choose parameter values, initial conditions values, and the solvers you'd like to compare. There are two tabs: one for single comparison and one for global comparison. You may choose parameters with the sliders and input boxes for each model. Then, run the comparison and the results will be generated.

Appendix D: Additional Figures

Phase Portraits

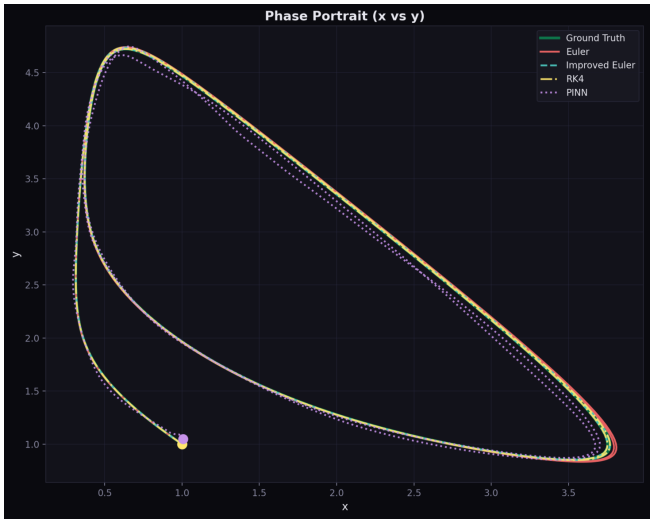


(a) Phase portrait at $\Delta t = 0.2$.

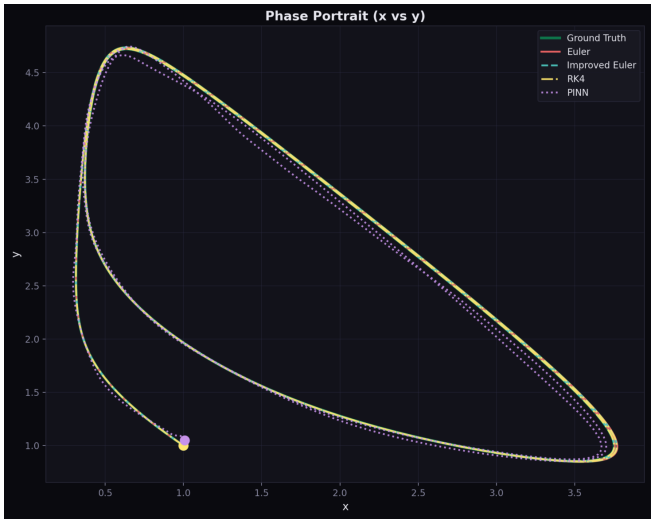


(b) Phase portrait at $\Delta t = 0.05$.

Figure 7: Phase portraits of the Brusselator for coarse and moderate time steps.



(a) Phase portrait at $\Delta t = 0.01$.



(b) Phase portrait at $\Delta t = 0.001$.

Figure 8: Phase portraits of the Brusselator for fine and reference time steps.

Time-Series (XY) Plots

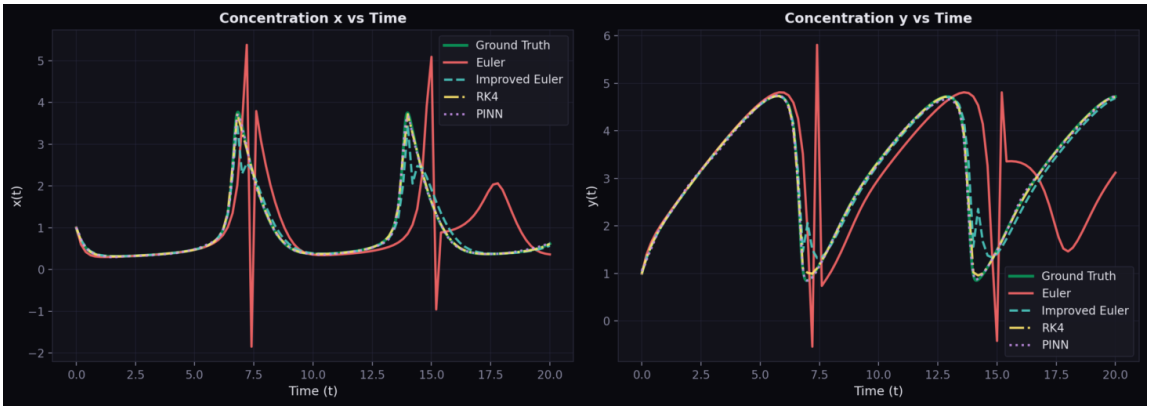


Figure 9: Time-series $x(t)$ and $y(t)$ at $\Delta t = 0.2$.

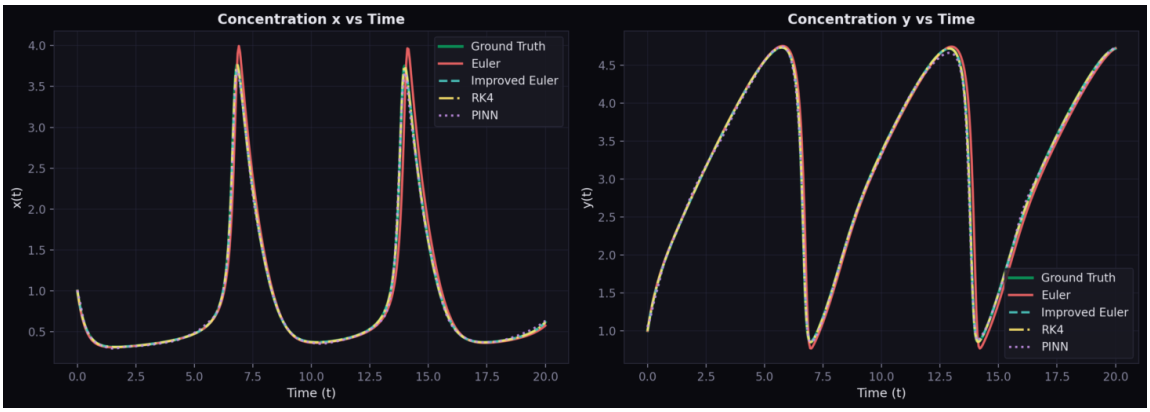


Figure 10: Time-series $x(t)$ and $y(t)$ at $\Delta t = 0.05$.

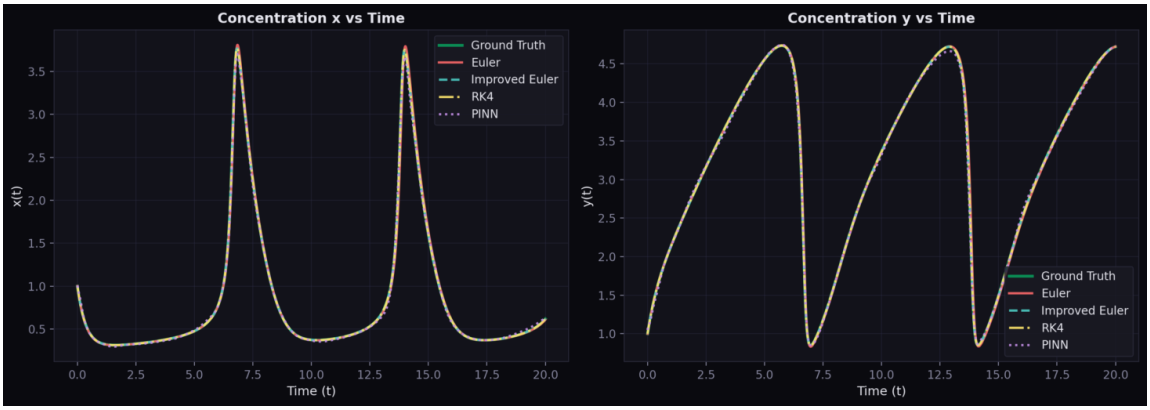


Figure 11: Time-series $x(t)$ and $y(t)$ at $\Delta t = 0.01$.

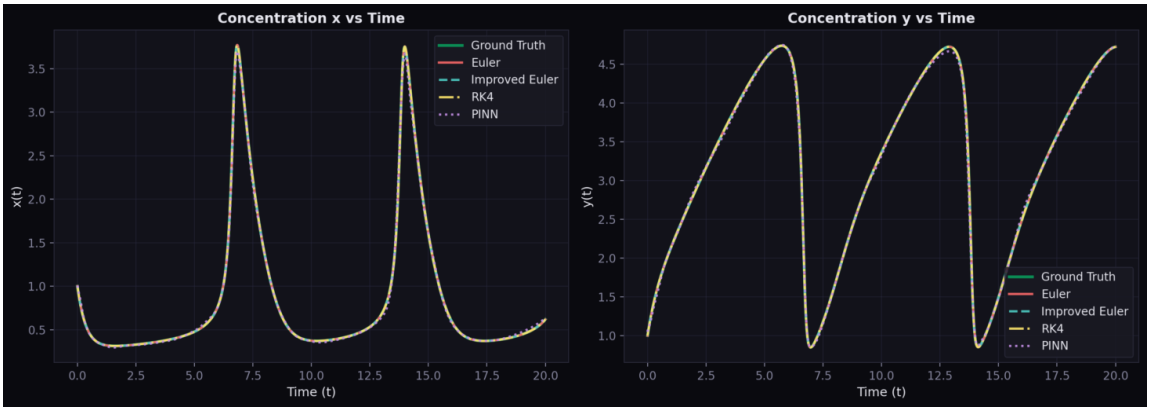


Figure 12: Time-series $x(t)$ and $y(t)$ at $\Delta t = 0.001$.

Global Error Distributions

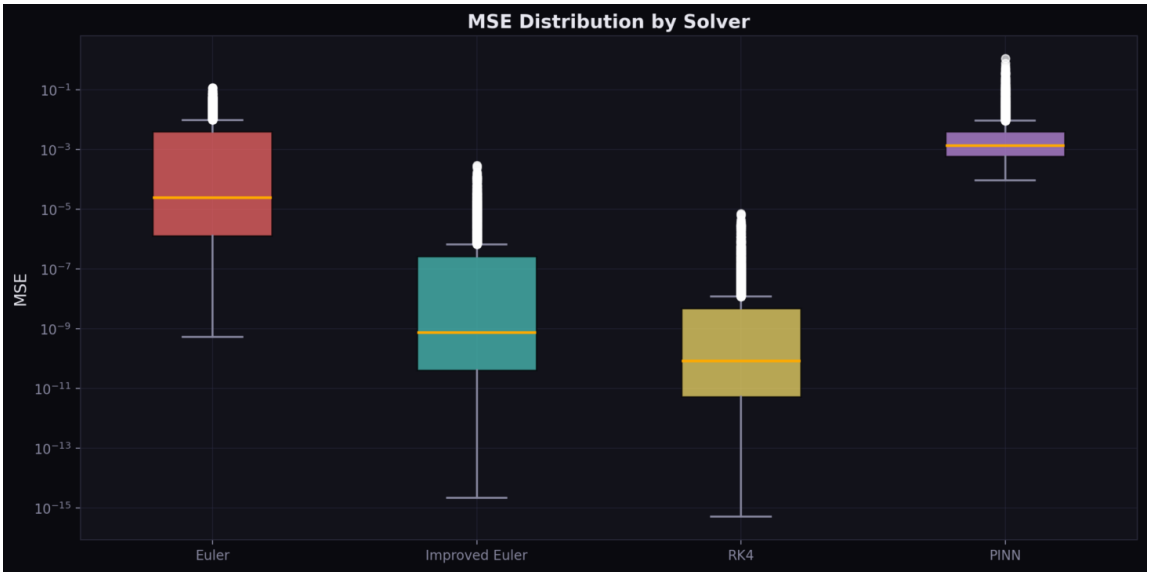


Figure 13: Distribution of MSE across randomized parameter sets.